Guia de Estudo

Rede Social em Python + SQLite

Explicação detalhada de cada arquivo do projeto

Visão Geral do Projeto

O projeto é uma rede social de terminal construída em Python, com persistência de dados usando SQLite. O código é dividido em 7 arquivos, cada um com uma responsabilidade bem definida.

O fluxo principal é:

- main.py recebe os comandos do usuário
- rede.py repassa as chamadas para o banco
- database.py executa as operações no SQLite
- init_db.py cria as tabelas na primeira execução
- usuario.py representa um usuário como objeto
- popular_banco.py insere dados de exemplo para demonstração
- teste_sistema.py valida que tudo funciona corretamente

usuario.py

Este é o arquivo mais simples do projeto. Ele define uma classe, que é uma forma de representar um conceito do mundo real no código — neste caso, um usuário da rede social.

O que é uma classe?

Uma classe é um molde para criar objetos. Ao definir a classe Usuario, estamos dizendo: todo usuário tem id, username, nome e idade. Quando o banco retorna dados de um usuário, o código cria um objeto Usuario com esses dados, o que facilita o acesso (usuario.nome ao invés de dados[2]).

O método __init__

```
def __init__(self, user_id, username, nome, idade):
    self.id = user_id
    self.username = username
    self.nome = nome
    self.idade = idade
```

O __init__ é o construtor — executado automaticamente ao criar um novo objeto. O self representa o próprio objeto sendo criado. Então quando database.py faz Usuario(*dados), ele passa os quatro valores de uma linha do banco direto para esse construtor.

O asterisco em Usuario(*dados) é um atalho do Python: desempacota uma tupla (4, 'joao123', 'João Silva', 20) como quatro argumentos separados.

O método __str__

```
def __str__(self):
    return f"ID: {self.id} | Username: {self.username} | ..."
```

Define o que aparece ao imprimir o objeto com print(). Sem ele, o Python mostraria algo como <usuario.Usuario object at 0x7f...>, que não é útil.

init_db.py

Responsável por criar a estrutura do banco de dados. Pense nele como a planta baixa do banco — define quais tabelas existem e quais colunas cada uma tem.

A tabela usuarios

```
CREATE TABLE IF NOT EXISTS usuarios (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    nome TEXT NOT NULL,
    idade INTEGER NOT NULL
)
```

Três detalhes importantes:

- IF NOT EXISTS: se a tabela já existir, o comando é ignorado. Por isso é seguro chamar criar_banco() toda vez que o programa inicia sem apagar os dados.
- AUTOINCREMENT: o SQLite gera o ID automaticamente. Primeiro usuário recebe ID 1, segundo recebe ID 2, e assim por diante.
- UNIQUE no username: o banco rejeita automaticamente qualquer tentativa de inserir um username que já existe.

A tabela amizades

```
CREATE TABLE IF NOT EXISTS amizades (
    usuario1 INTEGER NOT NULL,
```

```
    usuario2 INTEGER NOT NULL,
    FOREIGN KEY (usuario1) REFERENCES usuarios(id),
    FOREIGN KEY (usuario2) REFERENCES usuarios(id)
)
```

A tabela de amizades só armazena dois IDs, representando os dois lados do vínculo. As FOREIGN KEY são regras de integridade: garantem que os IDs em amizades sempre apontem para usuários que realmente existem no banco.

Como o SQLite funciona

O SQLite guarda tudo em um único arquivo (.db). Quando o código faz sqlite3.connect('rede_social.db'), o Python abre esse arquivo (ou cria se não existir). Toda operação de leitura ou escrita precisa de: abrir conexão → executar SQL → salvar (commit) → fechar conexão.

database.py

Este é o coração do projeto. Centraliza toda a lógica de comunicação com o banco. Cada função representa uma operação no SQLite.

conectar()

```
def conectar():
    return sqlite3.connect(DB_NAME)
```

Só um atalho para não repetir sqlite3.connect('rede_social.db') em toda função.

adicionar_usuario()

```
try:
    cursor.execute("INSERT INTO usuarios ...", (username, nome, idade))
    conn.commit()
    return True
except sqlite3.IntegrityError:
    return False
finally:
    conn.close()
```

O try/except captura o erro de username duplicado: quando a constraint UNIQUE é violada, o SQLite lança IntegrityError, e o código retorna False sem travar. O finally garante que a conexão seja fechada mesmo em caso de erro — importante para não deixar o banco travado.

remover_usuario()

Tem dois passos obrigatórios na ordem certa:

- Primeiro: apaga todas as amizades do usuário
- Depois: apaga o próprio usuário

Se fosse ao contrário, as amizades ficariam no banco apontando para um usuário que não existe mais — o que é chamado de registro órfão.

obter_id()

Função auxiliar interna: converte username em ID. Ela existe porque a tabela de amizades trabalha com IDs, mas o usuário digita usernames no terminal.

criar_amizade()

Tem três verificações antes de inserir:

```
if username1 == username2: return False    # autoamizade
if id1 is None or id2 is None: return False  # usuário inexistente
if cursor.fetchone(): return False         # amizade duplicada
```

A verificação de duplicata checa as duas ordens porque a amizade entre joao e maria é a mesma que entre maria e joao:

```
WHERE (usuario1 = ? AND usuario2 = ?)
   OR (usuario1 = ? AND usuario2 = ?)
```

Isso substitui a função _par() que outros projetos usam — aqui a lógica fica no SQL, não no Python.

remover_amizade()

```
removidas = cursor.rowcount
return removidas > 0
```

O rowcount indica quantas linhas foram afetadas pelo DELETE. Se for 0, a amizade não existia — então retorna False.

listar_amigos() e o JOIN

O JOIN é uma operação SQL que combina duas tabelas. Aqui ele cruza usuarios com amizades para trazer os dados completos dos amigos:

```
SELECT u.id, u.username, u.nome, u.idade
FROM usuarios u
JOIN amizades a
ON (
    (a.usuario1 = ? AND a.usuario2 = u.id)
    OR
    (a.usuario2 = ? AND a.usuario1 = u.id)
)
```

O ON define a condição do cruzamento — novamente checando as duas ordens possíveis, pois a amizade é bidirecional.

contar_amigos()

Usa COUNT(*) do SQL para contar quantas linhas na tabela amizades envolvem o user_id informado, seja na coluna usuario1 ou usuario2.

rede.py

A camada mais simples do projeto. A classe RedeSocial existe principalmente para cumprir o requisito de modularização: main.py nunca acessa database.py diretamente, sempre passa por RedeSocial.

```
class RedeSocial:
    def adicionar_usuario(self, username, nome, idade):
        return database.adicionar_usuario(username, nome, idade)
```

Cada método só repassa a chamada para database.py. Em projetos maiores essa camada teria regras de negócio próprias, mas aqui ela serve para separar claramente a interface do banco de dados — o que é uma boa prática de programação.

main.py

A interface do programa. Tudo que o usuário vê e digita passa por aqui.

O loop principal

```
while True:
    menu()
    opcao = input("Escolha uma opcao: ").strip()
    match opcao:
        case "1": adicionar_usuario(rede)
        ...
        case "7": break
```

O while True mantém o programa rodando até o usuário escolher sair (break). O match/case é uma estrutura do Python 3.10+ equivalente ao switch de outras linguagens — compara o valor de opcao com cada case e executa o bloco correspondente. O case _ é o caso padrão (equivalente ao else).

O .strip() no input() remove espaços em branco do começo e fim — evita que um espaço acidental cause opcao invalida.

criar_banco() no início

```
def main():
    criar_banco()  # sempre antes de qualquer operação
    rede = RedeSocial()
```

Garante que o banco e as tabelas existam antes de qualquer operação. Como usa CREATE TABLE IF NOT EXISTS, é seguro chamar sempre — não apaga dados existentes.

Validação de entrada

```
try:
    idade = int(input("Idade: "))
except ValueError:
    print("Idade invalida.")
    return
```

Protege contra o usuário digitar letras no campo de idade. Se int() receber algo que não é número, lança ValueError — o código captura e avisa sem travar.

popular_banco.py

Arquivo auxiliar para demonstração. Insere usuários e amizades de exemplo no banco antes da apresentação.

A função limpar_banco()

```
cursor.execute("DELETE FROM amizades")
cursor.execute("DELETE FROM usuarios")
cursor.execute("DELETE FROM sqlite_sequence WHERE name='usuarios'")
```

A terceira linha reseta o contador de IDs. Sem ela, se rodar o script duas vezes, o primeiro usuário da segunda rodada receberia ID 6 (continuando de onde parou), não ID 1. O sqlite_sequence é uma tabela interna do SQLite que guarda o último ID usado por cada tabela com AUTOINCREMENT.

Por que limpar antes de popular?

Se não limpar, rodar o script uma segunda vez tentaria inserir os mesmos usernames novamente, o que falharia silenciosamente por causa da constraint UNIQUE. Com a limpeza, o banco sempre fica no estado esperado independente de quantas vezes o script é rodado.

teste_sistema.py

Valida automaticamente as principais funcionalidades do sistema. Útil para verificar que tudo funciona antes da apresentação.

Como funciona a função checar()

```
def checar(descricao, condicao):
    if condicao:
        ok += 1
        print(f"v {descricao}")
    else:
        falhas += 1
        print(f"x {descricao}")
```

Recebe uma descrição e uma condição booleana. Se a condição for True, o teste passou. Se for False, falhou.

O uso de not nos testes negativos

```
resultado = rede.adicionar_usuario(usuario_teste, "Outro", 30)
checar("Bloqueio de username duplicado", not resultado)
```

Aqui esperamos que resultado seja False (porque o username já existe). O not False vira True, então o teste passa. Essa lógica de not é usada sempre que o teste verifica que algo foi corretamente bloqueado.

Limpeza ao final

```
rede.remover_usuario(usuario_teste)
rede.remover_usuario(amigo_teste)
```

Remove os dois usuários criados durante os testes para não poluir o banco. Sem isso, rodar o script uma segunda vez falharia no primeiro teste (cadastro) porque o username já existiria.

Glossário rápido

- SQLite: banco de dados leve que guarda tudo em um único arquivo .db, sem precisar instalar nada separado.
- SQL: linguagem para manipular bancos de dados. Os principais comandos usados no projeto são SELECT (buscar), INSERT (inserir), DELETE (remover) e COUNT (contar).
- cursor: objeto do Python que executa comandos SQL na conexão aberta.
- commit: confirma as alterações no banco. Sem commit, as mudanças são descartadas ao fechar a conexão.
- rowcount: número de linhas afetadas pelo último comando executado.
- JOIN: operação SQL que combina duas tabelas com base em uma condição.
- FOREIGN KEY: regra que garante que um ID em uma tabela aponta para um registro que realmente existe em outra tabela.
- IntegrityError: erro do SQLite quando uma regra do banco é violada (ex: username duplicado).
- try/except/finally: estrutura para tratar erros. try tenta executar, except captura o erro, finally executa sempre independente do resultado.
- match/case: estrutura do Python 3.10+ para comparar um valor com múltiplos casos (equivalente ao switch de outras linguagens).